

Word2vec

Aula 17 – Introdução ao Processamento de Linguagem Natural (PLN)

Eduardo Corrêa Gonçalves

18/05/2026

Sumário

Introdução

Word2vec SGNS – Skip-Gram with Negative Sampling

- Intuição do algoritmo

- A tarefa falsa

- Treinamento: um exemplo ilustrativo passo a passo

Treinamento: detalhes

- Função de perda

- Treinamento com gradiente descendente

Discussão

- Prós e contras do Word2vec

Introdução (1/6)

- **Vetores Semânticos (embeddings) Esparsos**
 - Na semana anterior, vimos a **forma básica** para representar uma palavra como um vetor semântico:
 - Os vetores são criados através da **matriz termo-contexto**.
 - Possuem **dimensão = $|V|$** (tamanho do vocabulário)
 - Tipicamente, **50.000** ou mais !!!!
 - Seus valores (TF-IDF ou frequência) são determinados em função da contagem de **coocorrência das palavras** com suas vizinhas.
 - **Desvantagem do método:** produz vetores **esparsos** e **longos**.

	[1]	[2]	[3]	[4]	[5]	[...]	[49999]	[50000]
hora	0	2	0	0	0	...	0	0

	[1]	[2]	[3]	[4]	[5]	[...]	[49999]	[50000]
certa	0	0	0	0	0	...	1	0

Introdução (2/6)

- Vetores Semânticos (embeddings) Densos
 - Nesta aula, conheceremos uma **representação mais poderosa**:
 - São os **embeddings curtos** (número de posições ***d*** entre 50-1000).
 - DETALHE: essas ***d*** dimensões não tem uma interpretação clara, como veremos !!!
 - E **densos** (maioria dos valores $\neq 0$).
 - Em vez de terem muitos zeros, são densos, **formados por valores reais** que podem ser positivos ou negativos.
 - Embeddings deste tipo (curtos e densos) demonstraram-se **mais eficazes** para resolver a maioria dos problemas de NLP.

Introdução (3/6)

- **Exemplo** – palavra “**hora**” representada em um embedding de 300 dimensões gerado pelo algoritmo Word2vec (implementação NILC (2017)).

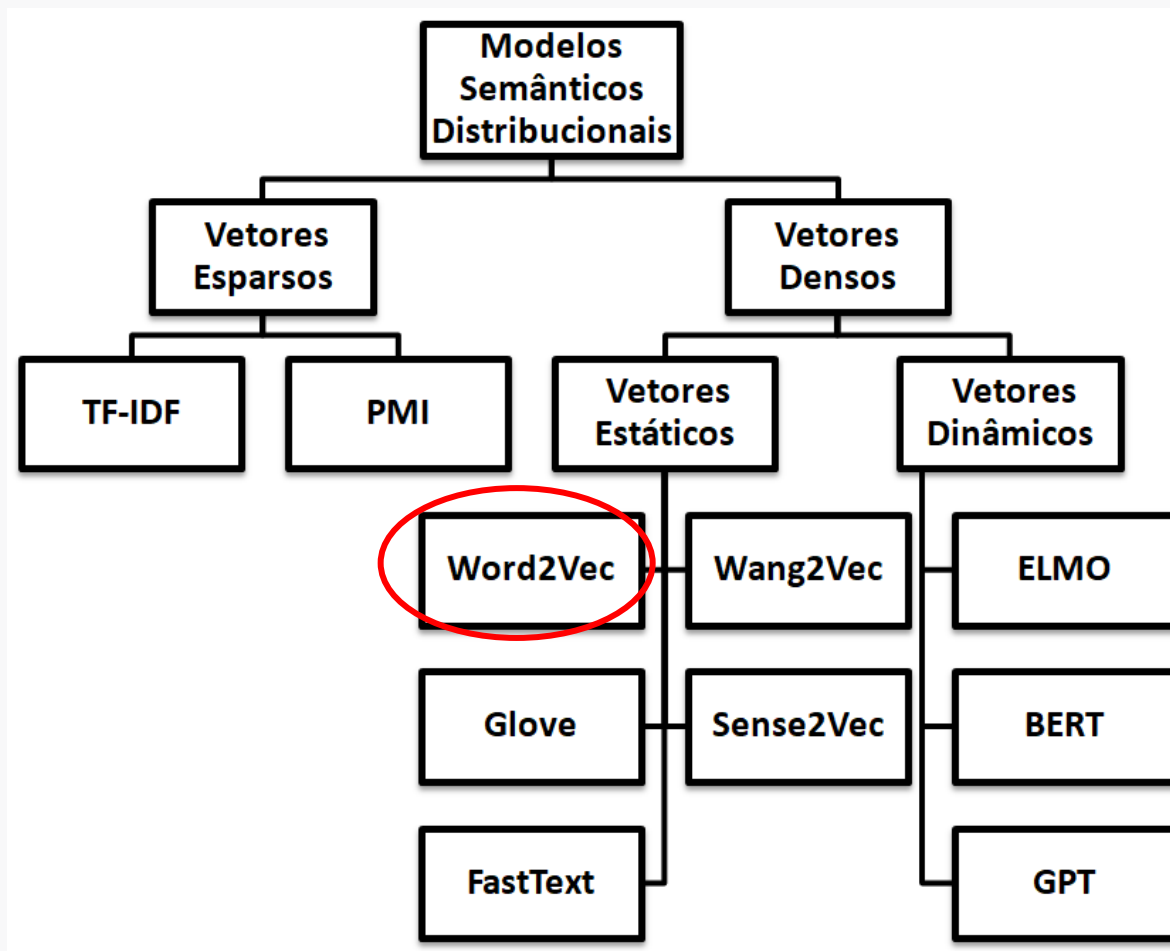
hora = [-0.062545, -0.126489, 0.113524, -0.073889, 0.006757, -0.027824, 0.286723, -0.010828, -0.032953, 0.348775, -0.212591, 0.223134, 0.136070, -0.256647, 0.072911, 0.021405, -0.058130, 0.191910, -0.113131, 0.072543, 0.030004, -0.056953, -0.115852, 0.029836, -0.156374, -0.249230, 0.112478, 0.126747, -0.004281, 0.014156, -0.407328, 0.108729, 0.349468, -0.035015, -0.328122, -0.051109, 0.233072, 0.244711, 0.108190, 0.297134, -0.024735, 0.127016, -0.336429, 0.049182, 0.155089, 0.093664, -0.121086, -0.182766, 0.118789, -0.002741, -0.031592, 0.052193, 0.081095, -0.183748, 0.036250, 0.388335, -0.240749, -0.047723, -0.078823, 0.156688, 0.122014, 0.589334, -0.021866, 0.185283, -0.018192, 0.077296, 0.030083, -0.398973, -0.241083, 0.038450, 0.206490, 0.435420, 0.011289, -0.019493, -0.119840, 0.155041, 0.104586, 0.434048, -0.140947, -0.038420, 0.518088, -0.158477, 0.140301, 0.155320, -0.020636, -0.149190, 0.069280, -0.247552, -0.115318, 0.103906, 0.173289, -0.111075, 0.061043, 0.180365, -0.092135, -0.105528, 0.360078, -0.303879, -0.054269, -0.121504, 0.132912, -0.047233, -0.052588, 0.018701, -0.005428, -0.007330, -0.021943, -0.000987, -0.105333, 0.168648, -0.243485, 0.020719, -0.246331, -0.297241, 0.041788, 0.105140, -0.289586, 0.023823, -0.122424, -0.064835, -0.158461, 0.068709, -0.205565, 0.173924, 0.126506, -0.244338, -0.052390, 0.174168, 0.338677, -0.287746, -0.020279, -0.022267, 0.199124, 0.058610, -0.107843, 0.057305, 0.104822, 0.062944, 0.231721, -0.244904, -0.015706, 0.173425, -0.010017, -0.227111, 0.137199, 0.222197, -0.154434, 0.189224, 0.250377, 0.396699, -0.026948, 0.053932, -0.246668, 0.108724, 0.095354, 0.184613, 0.077367, 0.080714, -0.000680, -0.071466, -0.324820, -0.163270, 0.142114, -0.176308, -0.040815, 0.277833, -0.025246, 0.053511, 0.409635, -0.213821, -0.142188, 0.384299, 0.130455, 0.324051, -0.025810, -0.086175, -0.207605, 0.018398, 0.193626, -0.160563, -0.280345, 0.413318, -0.126175, 0.034025, -0.356754, 0.078491, 0.164689, 0.037496, 0.147475, 0.086021, 0.190551, -0.296555, -0.348180, -0.122142, 0.032044, 0.189108, -0.346120, 0.272183, 0.128225, 0.173495, -0.135757, -0.063786, 0.125983, 0.215387, -0.112882, 0.043228, -0.060512, -0.021911, 0.014611, 0.083601, 0.019959, 0.346055, 0.014662, 0.098009, 0.094097, 0.017535, -0.075331, -0.095261, -0.218828, 0.101617, -0.235558, 0.193594, -0.027302, -0.003942, 0.039598, -0.080496, -0.067256, 0.299944, 0.143414, -0.136063, -0.136811, -0.205818, -0.068009, 0.059440, 0.135297, -0.135802, -0.078074, -0.186340, -0.101303, 0.081397, 0.039796, 0.052596, -0.071562, 0.075672, -0.194597, 0.058802, -0.361304, -0.041861, 0.011388, -0.070250, 0.132716, -0.193836, -0.148685, 0.039821, 0.124415, -0.011547, 0.319272, 0.163388, 0.280234, -0.146267, 0.323278, -0.065721, -0.091195, 0.189873, 0.160153, 0.053370, 0.223984, -0.151240, 0.163842, -0.071410, 0.039385, 0.066962, 0.329253, -0.067227, 0.298047, -0.055179, 0.081621, 0.102730, -0.131966, -0.272213, 0.083461, 0.224348, 0.002535, 0.003754, 0.267152, 0.076403, 0.074424, 0.145011, -0.153571, 0.041310, 0.122828, 0.064069, 0.075629, 0.195021, -0.181588, -0.140206, 0.093154, -0.127453, -0.153316, -0.021281]

Introdução (4/6)

- Por que embeddings curtos e densos funcionam melhor ?
 - Curiosamente, as razões não são totalmente compreendidas!
 - Algumas suposições (Jurafsky e Martin, 2025):
 - Representar palavras em vetores de 300 dimensões em vez de 50.000 facilita a vida dos algoritmos de aprendizado de máquina, já que eles poderão trabalhar com um **conjunto X** (variáveis de entrada) **bem menor**.
 - Vetores densos são mais “espertos” para capturar **palavras sinônimas** (*como veremos nessa aula*).

Introdução (5/6)

- Nesta aula apresentaremos o algoritmo **Word2vec SGNS** (*Skip-Gram with Negative Sampling*).
 - É um método que gera vetores **curtos**, **densos** e estáticos. Veja abaixo que há vários outros !!



Fonte: Seno et al. (2024)

Introdução (6/6)

- **IMPORTANTE**

- O Word2vec é da família dos “**estáticos**” porque aprende **1 embedding fixo** para cada palavra de **V** (vocabulário do problema).
 - **Exemplo:** nas 3 frases abaixo, “**banco**” está sendo usado com 3 significados diferentes, mas seu embedding Word2vec seria o mesmo.
 - “**Pode deixar, eu banco sua ideia**”.
 - “**Vamos sentar naquele banco?**”.
 - “**Preciso ir ao banco para sacar dinheiro**”.
- Porém, o estado-da-arte atual são os **embeddings dinâmicos**, gerados por métodos como **BERT** ...
 - ... onde o vetor para cada palavra é diferente para os diferentes contextos em que ela aparece.

Word2vec SGNS – Visão Geral (1/7)

- Word2vec SGNS – *Skip-Gram with Negative Sampling*
 - O modelo Word2vec SGNS é conceitualmente simples!
 - Não é *deep learning*.
 - Sua **ideia básica** é gerar os embeddings utilizando o truque da **tarefa falsa** (*fake task*):
 - Treina um modelo de regressão logística* (RL) para executar uma tarefa de classificação específica (*veremos qual*).
 - Entretanto, não estaremos de fato interessados na tarefa para qual o classificador foi treinado !!
 - O que faremos é usar os **pesos** aprendidos como **embeddings** das palavras !!

* Alguns autores descrevem como um modelo de rede neural de 1 camada escondida.

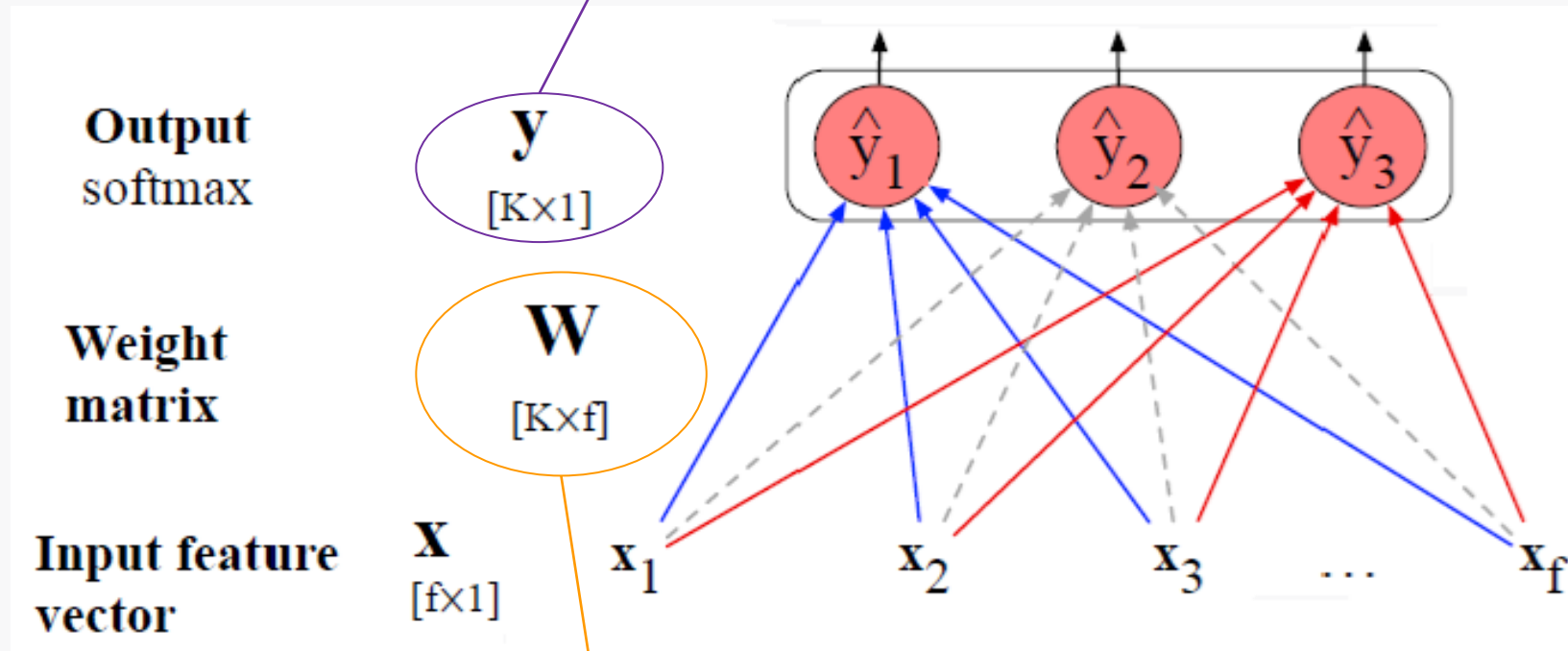
Word2vec SGNS – Visão Geral (2/7)

- A Tarefa Falsa de Classificação
 - Abaixo, a descrição da tarefa falsa p/ qual o modelo será construído:
 - Dada uma palavra-alvo w , o modelo deve dizer a probabilidade de cada palavra c em V (vocabulário do problema) ser uma palavra de contexto de w .
 - **Relembrando:** palavra de contexto é aquela que frequentemente coocorre com a palavra-alvo dentro de uma dada janela (ex.: 3 palavras à direita e à esquerda)
 - **Exemplo:** $w = \text{"damasco"}$
 - As probabilidades de saída do modelo devem ser **maiores** para palavras como **"geleia"** e **"licor"**
 - E **menores** para palavras como **"canguru"** e **"estatística"**

Word2vec SGNS – Visão Geral (3/7)

- A Tarefa Falsa de Classificação

Não vamos usar o modelo para a tarefa que ele foi treinado !!!



- O que faremos no final do treinamento é **pegar os pesos** !
- Os **pesos** serão os **embeddings**

Word2vec SGNS – Visão Geral (4/7)

- A Tarefa Falsa de Classificação
 - Na sentença abaixo, considere:
 - A palavra-alvo sendo “**apricot**” (damasco)
 - E uma janela de contexto de ± 2 palavras

...	lemon,	a	[	tablespoon	of	apricot	jam,	a]	pinch	...
				c1		c2	w	c3		c4

Word2vec SGNS – Visão Geral (5/7)

- O objetivo da Tarefa Falsa é treinar um classificador que:
 - Dada uma tupla (w, c) contendo uma **palavra alvo** w fazendo par com uma **palavra de contexto candidata** c ...
 - ... o classificador deverá retornar a **probabilidade** de que c seja uma **palavra de contexto real**.
 - Tem que ser maior para (“**apricot**”, “**jam**”) (damasco, geléia)
 - Do que para (“**apricot**”, “**aardvark**”) (damasco, porco-da-terra)
- **Notação:**
 - $P(+|w,c)$ → probabilidade de c ser, de fato, uma palavra de contexto
 - $P(-|w,c) = 1 - P(+|w,c)$ → probabilidade de não ser

...	lemon,	a	[tablespoon	of	apricot	jam,	a]	pinch	...
		c1		c2	w	c3		c4	

Word2vec SGNS – Visão Geral (6/7)

- Como a probabilidade P é computada?
 - $P(+|w,c) \rightarrow$ probabilidade de c ser uma palavra de contexto
 - $P(-|w,c) = 1 - P(+|w,c) \rightarrow$ probabilidade de não ser
- A intuição é basear essa probabilidade em **similaridade de embeddings**.
 - Uma palavra c é provavelmente de contexto de uma palavra w se o embedding de c é parecido com o embedding de w .
 - O Word2vec baseia-se na intuição de que os vetores são similares se possuírem **valor alto** para o **produto escalar**.
 - $\text{sim}(w, c) \approx w \cdot c$
 - *(reveja a aula anterior para detalhes sobre o produto escalar)*

Word2vec SGNS – Visão Geral (7/7)

- Como a probabilidade P é computada?
 - $\text{sim}(w, c) \approx w \cdot c$
 - O produto escalar não é probabilidade, mas um valor entre $-\infty$ e $+\infty$.
 - Então, para converter para probabilidade, utiliza-se a **função sigmoide**.

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

- Sendo assim, a probabilidade de c ser palavra de contexto real é:

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

- Por consequência, a probabilidade de que não seja é:

$$P(-|w, c) = 1 - P(+|w, c) = \sigma(-c \cdot w) = \frac{1}{1 + \exp(c \cdot w)}$$

Word2vec SGNS – Exemplo Ilustrativo (1/16)

- O Word2vec SGNS executa os seguintes passos:
 1. Geração a base de dados de treinamento
 - Identificação de todos os pares (w , c) do corpus (**palavra-alvo**, **palavra vizinha de contexto**), que serão rotulados como exemplos positivos.
 - Seleção aleatória de outras palavras no vocabulário para formar exemplos negativos*.
 2. Treinamento um classificador que diferencia os dois casos acima usando regressão logística.
 3. Ao final do treinamento, usar os pesos aprendidos como embeddings.
- *Os próximos slides ilustram o funcionamento desses passos omitindo detalhes matemáticos. O exemplo foi retirado de Allamar (2019).*

* daí vem o nome *skip gram with negative sampling*

Word2vec SGNS – Exemplo Ilustrativo (2/16)

- Passo 1a: Gerar o BD de treinamento – inserir instâncias positivas
 - **Todos** os pares (*w*, *c*) do corpus são automaticamente obtidos e inseridos no BD de treino como instâncias positivas (classe 1).
- **Exemplo ao lado** – mostra como seria feito para o trecho destacado da frase abaixo adotando janela de ± 2 palavras de contexto.
 - “*Thou shalt not make a machine in the likeness of a human mind*”

w	c	classe
...	...	...
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1
...	...	...

Word2vec SGNS – Exemplo Ilustrativo (3/16)

- **Passo 1b**: Gerar o BD de treinamento – inserir instâncias negativas
 - Selecionar aleatoriamente outras palavras no vocabulário para obter instâncias negativas (classe 0).
- **Exemplo** – para cada instância positiva, geramos k exemplos negativos.
 - No exemplo ao lado $k = 2$
- Os exemplos negativos precisam ser:
 - Palavras que **são do vocabulário** mas que **não coocorrem com a palavra-alvo**.
 - Detalhes sobre o processo em Jurafsky e Martin (2025)

w	c	classe
...	...	...
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	fingerlonger	0
not	make	1
not	plumbus	0
not	bat	0
...	...	...

Word2vec SGNS – Exemplo Ilustrativo (4/16)

- Passo 1: Gerar o BD de treinamento
 - Obs1: veja que uma mesma palavra pode estar no BD na coluna “**w**” (palavra alvo) ou na coluna “**c**” (palavra de contexto)

w	c	classe
...	...	...
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
...	...	...
make	not	1
make	likeness	0
make	jay	0
...	...	...

Word2vec SGNS – Exemplo Ilustrativo (5/16)

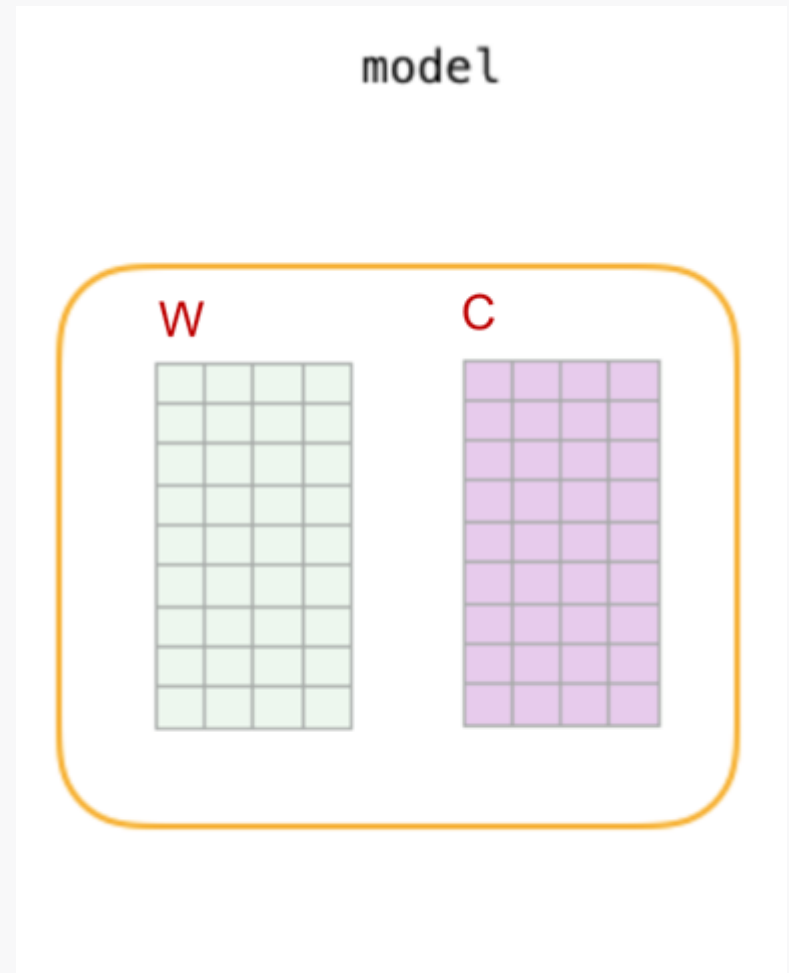
- Passo 1: Gerar o BD de treinamento
 - Obs2: Veja também que o BD de treino é **gerado** de **forma totalmente automática**.
 - Não é necessário que uma pessoa rotule manualmente os exemplos como 1 ou 0.

w	c	classe
...	...	...
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	fingerlonger	0
...	...	...
make	not	1
make	likeness	0
make	jay	0
...	...	...

Word2vec SGNS – Exemplo Ilustrativo (6/16)

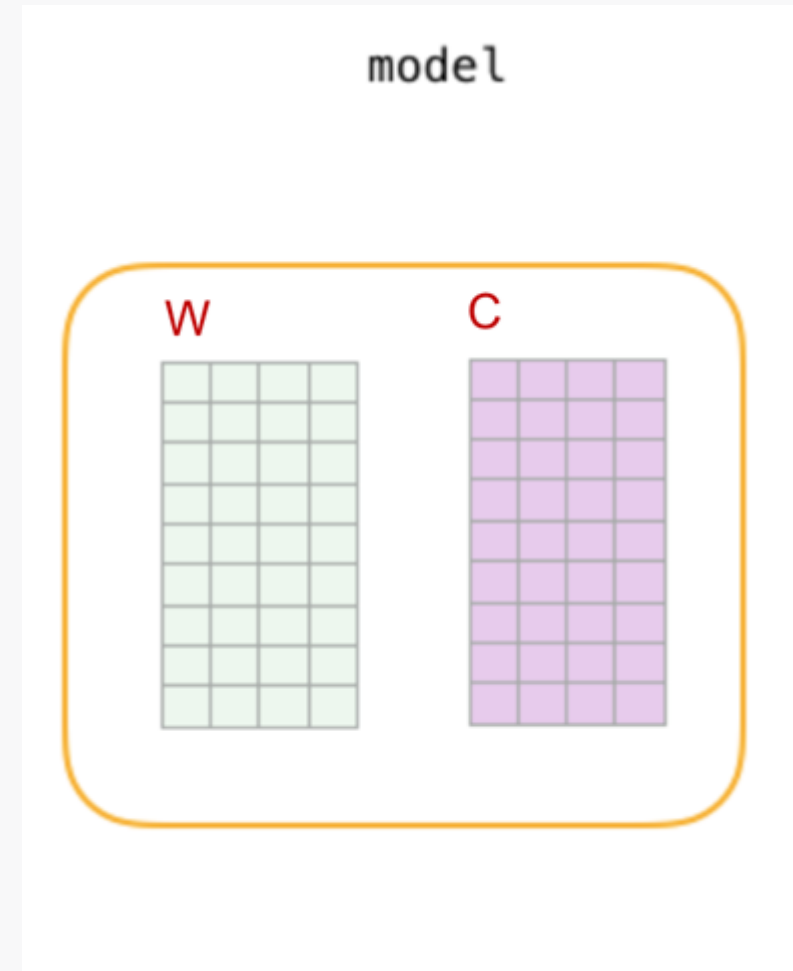
- Passo 2: Treinar classificador que diferencia os 2 casos
 - O Word2vec começa **gerando 2 matrizes** **W** (Embedding) e **C** (Contexto)
 - Ao final do treinamento, cada uma das matrizes conterá um embedding para cada palavra do vocabulário (!!!!!)

w	c	classe
...	...	...
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
...	...	...
make	not	1
make	likeness	0
make	jay	0
...	...	...



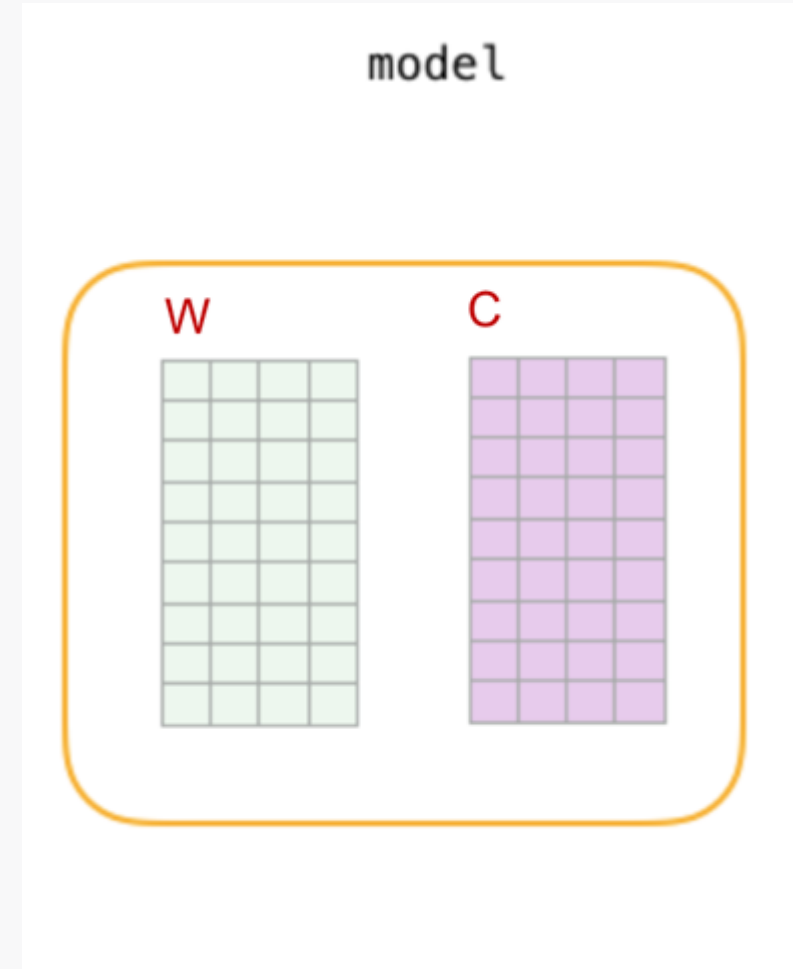
Word2vec SGNS – Exemplo Ilustrativo (7/16)

- Treinamento Passo 2a: inicializar as matrizes W e D
 - W contém os embeddings das palavras-alvo
 - C contém os embeddings das palavras de contexto
 - Ambas as matrizes possuem **dimensão $|V| \times d$**
 - Cada linha representa 1 palavra do vocabulário, por isso são $|V|$ linhas.
 - O número de colunas d é o tamanho do embedding
 - Entre 50 e 1000, sendo 300 o tamanho mais comum.



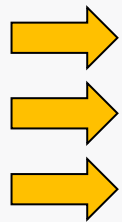
Word2vec SGNS – Exemplo Ilustrativo (8/16)

- Treinamento Passo 2a: inicializar as matrizes W e D
 - Ambas são **inicializadas** com **valores aleatórios**.
 - Entre -1 e 1 por exemplo

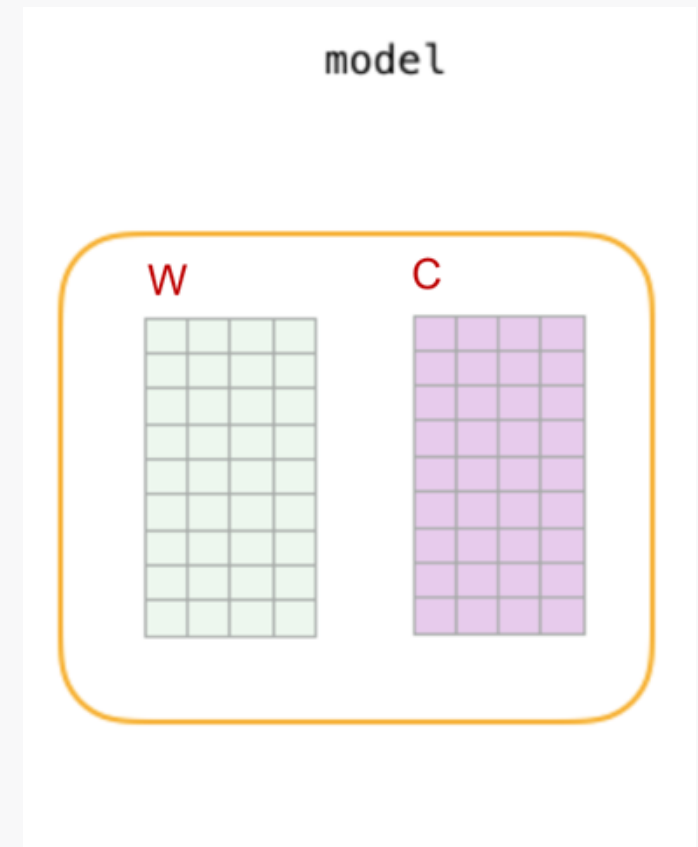


Word2vec SGNS – Exemplo Ilustrativo (9/16)

- Treinamento Passo 2b: Ajuste iterativo dos embeddings
 - Percorre-se todo o BD. A cada iteração, um exemplo positivo específico e os seus exemplos negativos associados serão tratados.
 - Na figura abaixo os exemplos tratados são:
 - **Positivo**: (“not”, “thou”)
 - **Negativos**: (“not”, “aaron”), (“not”, “taco”)



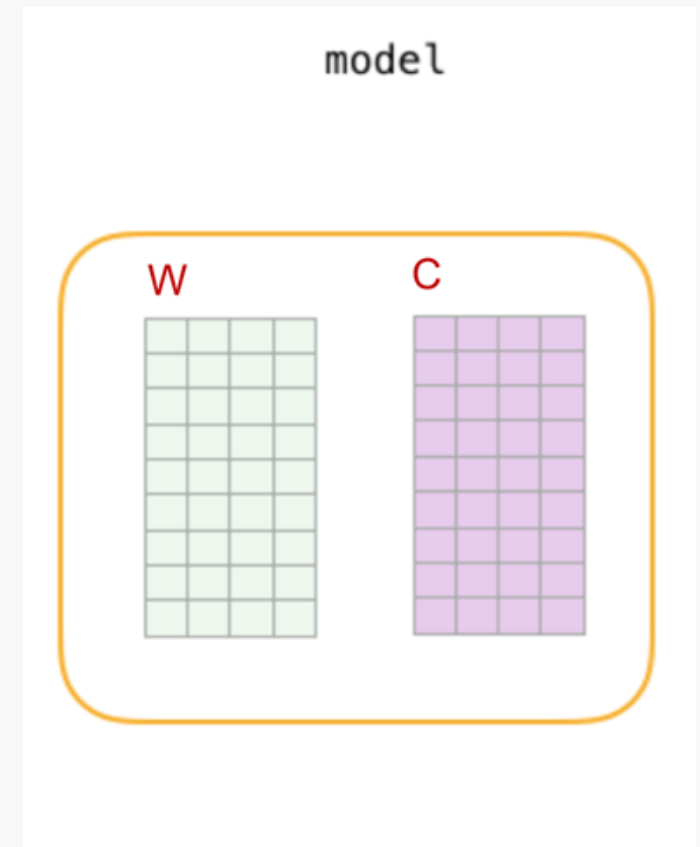
w	c	classe
...	...	...
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
not	bat	0
...	...	...



Word2vec SGNS – Exemplo Ilustrativo (10/16)

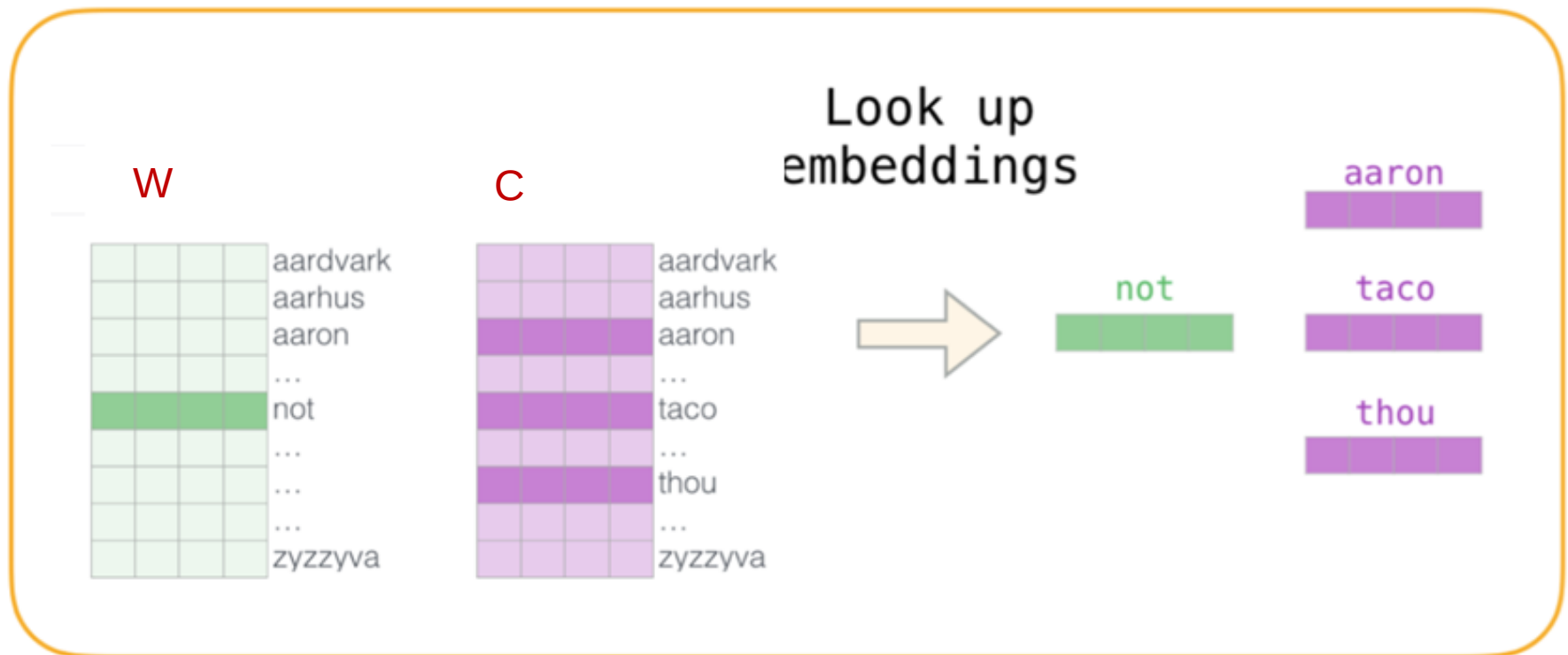
- Treinamento Passo 2b: Ajuste iterativo dos embeddings
 - Veja que há **4 palavras** envolvidas nos 3 exemplos:
 - “not” → a palavra-alvo da iteração corrente
 - “thou” → a palavra real de contexto
 - “aaron” e “taco” → as palavras negativas de contexto

w	c	classe
...	...	...
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
not	bat	0
...	...	...



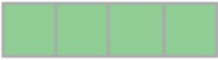
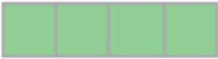
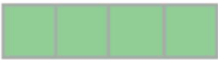
Word2vec SGNS – Exemplo Ilustrativo (11/16)

- Treinamento Passo 2b: Ajuste iterativo dos embeddings
 - Vamos então selecionar os embeddings dessas palavras nas matrizes.
 - Para a palavra-alvo (“not”), selecionamos em W
 - Para as palavras de contexto “thou”, “aaron” e “taco”, selecionamos em C .



Word2vec SGNS – Exemplo Ilustrativo (12/16)

- Treinamento Passo 2c: Computar a similaridade dos embeddings
 - Computa-se o **produto escalar** do embedding da palavra-alvo com os embeddings de cada palavra de contexto.
 - O valor refletirá a similaridade entre esses embeddings.
- Treinamento Passo 2d: Aplicar a Sigmoid
 - Em seguida, aplicamos a função Sigmoid para **converter** os valores computados **para probabilidades**.

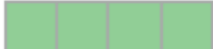
input word	output word	target	input • output	sigmoid()
not 	thou 	1	0.2	0.55
not 	aaron 	0	-1.11	0.25
not 	taco 	0	0.74	0.68



Word2vec SGNS – Exemplo Ilustrativo (13/16)

- Treinamento Passo 2e: Calcular o erro
 - Note que ao aplicar a Sigmoid, isso significa que modelo fez uma predição
 - Se $\text{prob} > 0.5$, modelo estimou como **classe 1**.
 - Caso contrário como **classe 0**.
 - Podemos então, **calcular o erro**:
 - **valor real - valor estimado**



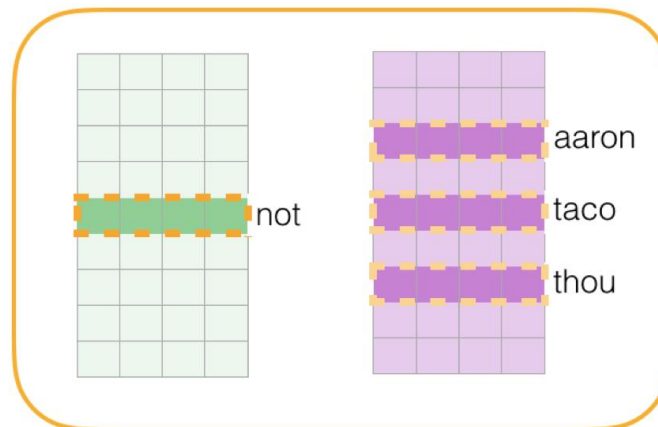
input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68

Word2vec SGNS – Exemplo Ilustrativo (14/16)

• Treinamento Passo 2f: Ajustar os embeddings

- O erro é utilizado para ajustar os valores dos embeddings de “not” (em W) e “thou”, “aaron” e “taco” (em C).
- Isso é feito de modo que os embeddings de “not” e “thou” se aproximem
- E o embedding de “not” se afaste de “aaron” e “taco”.

input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68

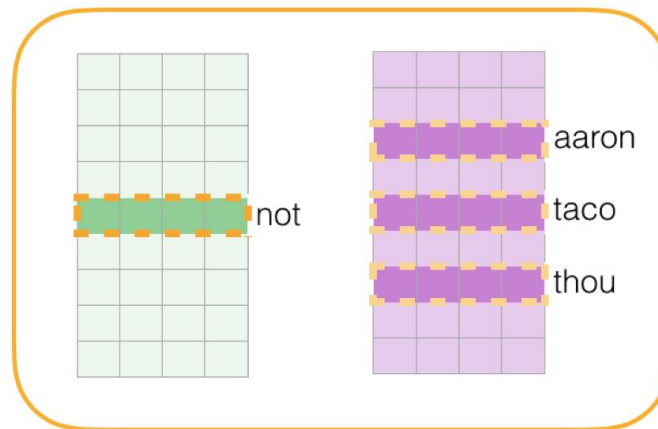


Update
Model
Parameters

Word2vec SGNS – Exemplo Ilustrativo (15/16)

- Passo 2b: (treinamento) Ir para a próxima iteração
 - Isso conclui uma iteração específica de uma dada época !!!
 - A seguir, vamos para a próxima iteração, pegando o exemplo positivo seguinte e os seus exemplos negativos associados.
 - Os embeddings serão sucessivamente aprimorados diversas vezes (diversas épocas)

input word	output word	target	input • output	sigmoid()	Error
not	thou	1	0.2	0.55	0.45
not	aaron	0	-1.11	0.25	-0.25
not	taco	0	0.74	0.68	-0.68



Update
Model
Parameters

Word2vec SGNS – Exemplo Ilustrativo (16/16)

- **Escolhendo os embeddings finais de cada palavra**
 - Ao fim de tudo, o modelo é jogado fora e pegaremos apenas os embeddings gerados.
 - Porém, vimos que o algoritmo salva 2 embeddings p/ cada palavra de V :
 - **Target embedding** (ou **input**) na matriz W .
 - **Context embedding** (ou **output**) na matriz C .
 - Com qual delas ficaremos no final? Algumas alternativas:
 - É comum simplesmente jogar a matriz C fora e **ficar com a W** .
 - Ou **adicionar as duas matrizes** e usar os embeddings da matriz resultante.

Detalhes do Treinamento (1/6)

- O Word2vec treina um classificador de regressão logística (RL)
- Sendo assim, emprega 2 componentes no treinamento:
 1. Uma **função de perda** para avaliar o quão distante a classe estimada está da real.
 - Lembre-se que no modelo:
 - A **classe 1** corresponde a “**c é palavra de contexto de w** ”
 - A **classe 0** corresponde a “**c não é palavra de contexto de w** ”
 2. Um **algoritmo** para otimizar a função de perda
 - Como sabemos, a RL usa o **Gradiente Descendente Estocástico**

Detalhes do Treinamento (2/6)

- Componente 1 do Treino – Função de Perda
 - O objetivo do SGNS é:
 - Maximizar a similaridade dos pares positivos (w , c_{pos})
 - Minimizar a similaridade dos pares negativos (w , c_{neg})
 - Considerando-se um par específico (w , c_{pos}) com seus k exemplos negativos c_{neg1} , c_{neg2} , ... c_{negk} ...
 - ... Esses 2 objetivos podem ser expressos como a seguinte **função de perda** a ser **minimizada** (por isso o sinal -)

$$\begin{aligned}
 L &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right] \\
 &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\
 &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right] \\
 &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]
 \end{aligned}$$

Detalhes do Treinamento (3/6)

- Função de Perda

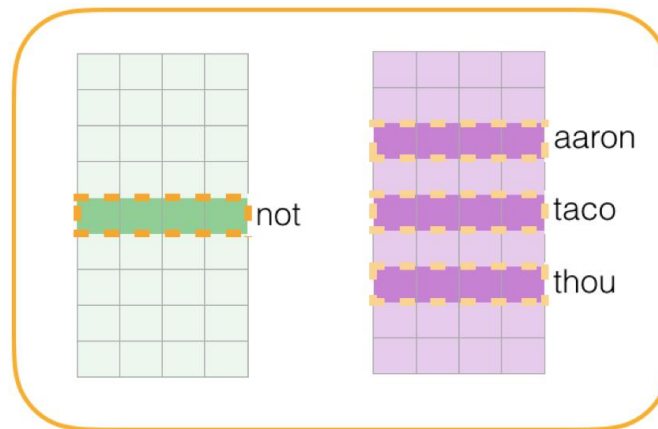
$$L = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

- O 1º termo expressa que:
 - Desejamos que o classificador associe uma probabilidade alta para a palavra de contexto real c_{pos} **ser vizinha** de w .
- O 2º termo expressa que:
 - Desejamos atribuir para cada palavra negativa c_{neg} uma alta probabilidade de **não ser vizinha** de w .
- Em outras palavras:
 - Desejamos maximizar o produto escalar de w com a palavra de contexto real ...
 - ... e minimizar os produtos escalares com as k palavras não-vizinhas

Detalhes do Treinamento (4/6)

- **Gradiente Descendente Estocástico (GD)**
 - A função de perda L é minimizada utilizando o GD.
 - Recordando nosso exemplo ilustrativo, no passo da figura abaixo:
 - os embeddings de “not” e “thou” seriam aproximados
 - E o embedding de “not” seria afastado dos de “aaron” e “taco”.

input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68



Update
Model
Parameters

Detalhes do Treinamento (5/6)

- **Gradiente Descendente Estocástico (GD)**
 - Para calcularmos o gradiente, precisamos das **derivadas** da função de perda ...

$$L = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

- ... com **relação** aos **diferentes embeddings**.
 - Aqui estão elas:

$$\frac{\partial L}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w$$

$$\frac{\partial L}{\partial c_{neg}} = [\sigma(c_{neg} \cdot w)]w$$

$$\frac{\partial L}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w)]c_{neg_i}$$

Detalhes do Treinamento (6/6)

- **Gradiente Descendente Estocástico (GD)**
 - As equações de atualização de uma iteração t para a $t+1$ no GD são:

$$\begin{aligned} \mathbf{c}_{pos}^{t+1} &= \mathbf{c}_{pos}^t - \eta [\sigma(\mathbf{c}_{pos}^t \cdot \mathbf{w}^t) - 1] \mathbf{w}^t \\ \mathbf{c}_{neg}^{t+1} &= \mathbf{c}_{neg}^t - \eta [\sigma(\mathbf{c}_{neg}^t \cdot \mathbf{w}^t)] \mathbf{w}^t \\ \mathbf{w}^{t+1} &= \mathbf{w}^t - \eta \left[[\sigma(\mathbf{c}_{pos}^t \cdot \mathbf{w}^t) - 1] \mathbf{c}_{pos}^t + \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i}^t \cdot \mathbf{w}^t)] \mathbf{c}_{neg_i}^t \right] \end{aligned}$$

- Maiores informações podem ser obtidas em Jurafsky e Martin (2025).

Prós e Contras do Word2vec (1/2)

- **Vantagens:**
 - (1) Treinamento **rápido**.
 - (2) **Não requer** uma **base rotulada** para ser treinado.
 - (3) **Disponível** em **vários idiomas**, incluindo Português (NILC, 2017)
 - (4) **Disponível** em **vários pacotes** e toolkits.
 - (5) A partir dos embeddings de palavras, podemos **gerar facilmente embeddings para frases, parágrafos e até documentos inteiros**.
 - Por exemplo o embedding da expressão “**hora certa**” pode ser gerado como:
 - A **média** dos embeddings de “**hora**” e “**certa**”
 - A **soma** dos embeddings de “**hora**” e “**certa**”

Prós e Contras do Word2vec (2/2)

- **Desvantagens:**

- (1) Foi o **1º método** para geração de embeddings estáticos e, atualmente, já é **considerado datado**.
 - Porém, devido a sua importância, ele é apresentado em vários livros, como é o caso do livro do Jurafsky.
 - Há outros algoritmos que geram embeddings estáticos que vieram depois. Eles aprimoraram o Word2vec e contornaram algumas de suas deficiências:
 - **fasttext**
 - **glove**
 - **wang2vec**
- (2) **Gera 1 embedding fixo** para cada palavra, ao contrário de métodos como **BERT**, que geram embeddings dinâmicos.
- (3) **Amplifica vieses e preconceitos** da sociedade. (*[mais detalhes no livro do Jurafsky](#)*)

A Nossa Tabela de Técnicas !!!

- Tabela: propriedades das técnicas apresentadas em nosso curso

Técnica	Principais Aplicações	Dimensão	Paradigma
<i>n</i>-Gramas de Caracteres	<i>features</i> para modelos de identif. de plágio, detecção de idioma, atribuição de autor etc.; comparação de strings.	Morfológica	Simbólico
Bag of Words (BoW)	produzir <i>features</i> p/ Naive Bayes e algoritmos de classificação, <i>clustering</i> , regressão etc.	Morfológica	Simbólico
Naïve Bayes Multinomial	análise de sentimentos, classificação de texto	Morfológica	Estatístico
Perceptron	classificação binária de texto	Fonética a Semântica	Estatístico, Neural
Engenharia de características	produzir <i>features</i> para algoritmos de classificação, <i>clustering</i> , regressão etc.	Fonética a Semântica	Simbólico, Estatístico
StemBR, RSLP	stemming	Morfológica	Simbólico
TF-IDF	produzir <i>features</i> para algoritmos de classificação, <i>clustering</i> , regressão etc.	Morfológica	Estatístico
WordNet	análise semântica, desambiguação de sentido das palavras, similaridade, classificação etc.	Semântica	Simbólico
Word2vec, Tok2Vec	geração de <i>features</i> para classificação, regressão e segmentação de textos, entre outras aplicações	Semântica	Neural

Referências

- ALAMMAR, J. **The Illustrated Word2vec**. Github.io, 2019. Disponível em: < <https://jalammar.github.io/illustrated-word2vec> >. Acesso em: 13 fev. 2025.
- JURAFSKY, D.; MARTIN, J. H. Vector Semantics and Embeddings. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. (Jan. 12, 2025 draft). [s.l.]: [s.n.], 2024. cap 6, p. 117 – 128. Disponível em: < <https://web.stanford.edu/~jurafsky/slp3/> >. Acesso em: 12 fev. 2025.
- NILC, **Repositório de Word Embeddings do NILC**. nilc.icmc.usp.br, 2017. Disponível em: < <http://nilc.icmc.usp.br/nilc/index.php/repositorio-de-word-embeddings-do-nilc> >. Acesso em: 12 fev. 2025.
- SENO, E. R. M. *et al.* Semântica distribucional. *In*: CASELI, H. M.; NUNES, M. G. V. **Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português**. 3. ed. [s.l.]: BPLN, 2024. cap 10, p. 13 – 22. Disponível em: < <https://brasileiraspln.com/livro-pln/3a-edicao/parte-significado/cap-semantica-distribucional/cap-semantica-distribucional.html> >. Acesso em: 12 fev. 2025.